

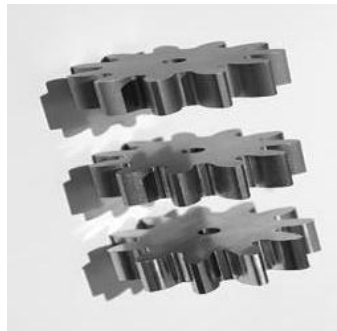
Transfer Learning for 3D-Printed Image Classification

Yuval Shaanan, Amit Shrager

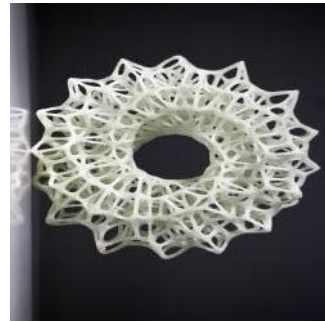
Introduction and Motivation

This project investigates the use of deep learning and transfer learning for automated classification of images as either 3D-printed or non-3D-printed.

<https://huggingface.co/datasets/cmudrc/3d-printed-or-not>



non-3D-printed



3D-printed

The motivation is to evaluate how well pretrained CNNs can identify visual characteristics associated with 3D printing while reducing the computational cost and training time compared with training a model from scratch.

The study compares **VGG**, **ResNet**, and **DenseNet** models to identify which architecture provides the best performance

Dataset and preprocessing

The dataset initially contained 51,520 images. After filtering, 51,488 images with a resolution of 256×256 pixels were retained, while 32 images were excluded. The resulting dataset contained 25,728 of non-3D-printed images and 25,760 3D-printed images, providing a nearly balanced class distribution.

Processing the full dataset of 51,488 images proved to be too computationally expensive. Therefore, Training was restricted to a balanced subset of 1,500 images per class (3,000 images in total). This subset proved sufficient to achieve strong classification performance, with test accuracy exceeding 92%. Increasing the subset size beyond 3,000 images produced no meaningful improvement in performance, indicating that the reduced sample was adequate to characterize the classification task.

The dataset was split into training, validation, and test sets using an 80/10/10 ratio.

The dataset was found to already contain a high degree of augmentation, with multiple transformed variants of the same underlying samples present in the original dataset. We found that additional augmentation was unnecessary.

Model and training description

All models used the same training setup: batch size of 16, Adam optimizer, binary cross-entropy loss, up to 3 epochs per stage. All three architectures were trained under an identical protocol, so that differences in performance reflect the base model rather than the

surrounding setup. Each takes $256 \times 256 \times 3$ RGB input with ImageNet pretrained weights, and each image is normalized using the architecture's own preprocess_input function. A common classification head was appended to every backbone: global average pooling, a 256-unit ReLU layer, dropout of 0.5, and a single sigmoid output.

Training used the Adam optimizer with binary cross-entropy loss, a batch size of 16, and up to three epochs per stage. The best checkpoint by validation accuracy was retained for each stage, and a fixed seed (42) was used.

Each model was trained in two stages.

Stage 1 - all pretrained layers were frozen and only the head was trained at a learning rate of $1e-3$.

Stage 2 -we unfroze the top layers and training continued at a much lower learning rate of $1e-5$, so the pretrained features could be adjusted slightly rather than overwritten. The batch-normalization layers in ResNet50 and DenseNet121 were left frozen, because updating them with such small batches would have spoiled the statistics they learned from ImageNet.

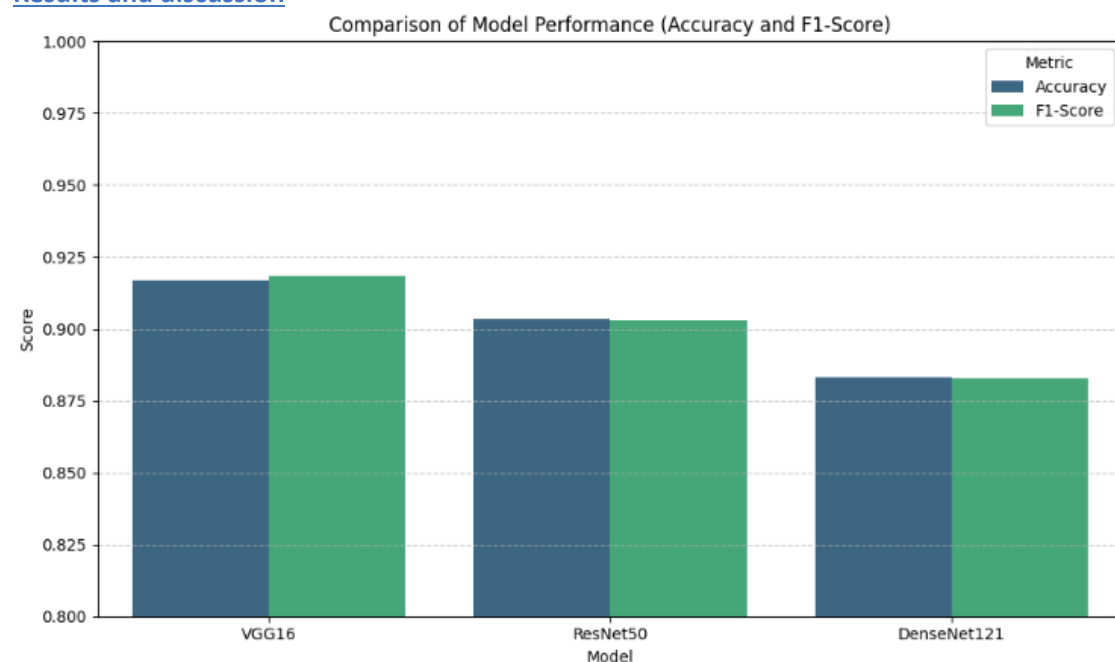
VGG- top 4 layers were unfrozen.

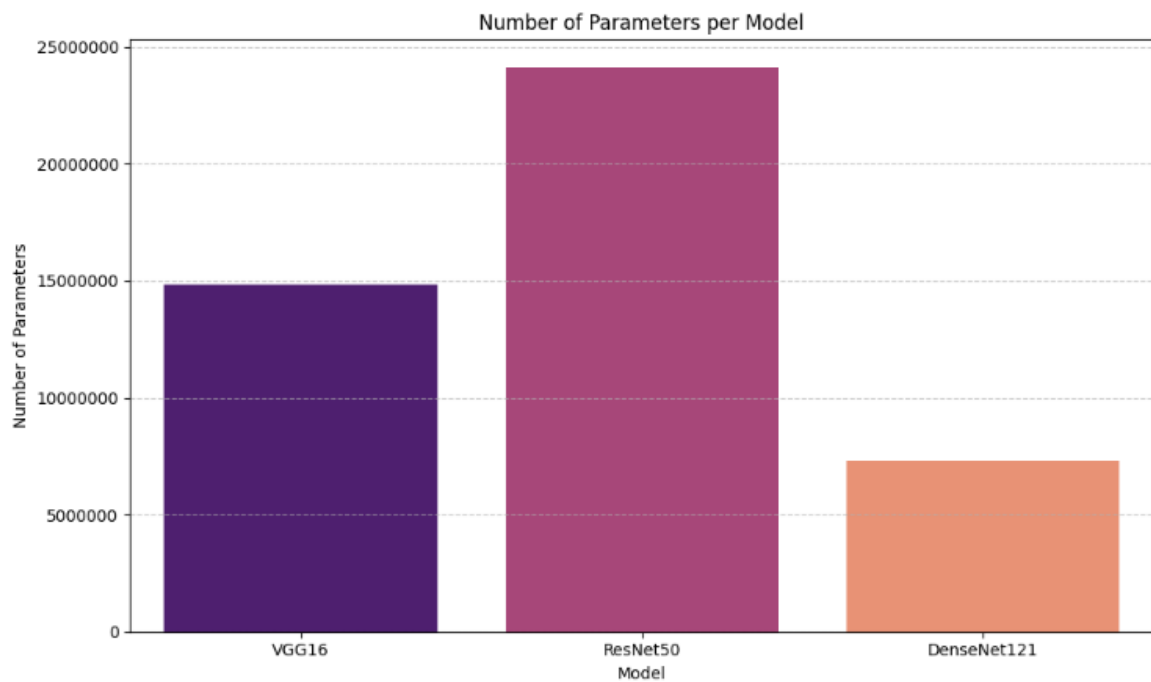
ResNet- top 20 layers were unfrozen

DenseNet121- top 40 layers were unfrozen.

The number of unfrozen layers differs because we wanted to fine tune only the top of the model. The difference corresponds to a different count in each architecture. Keras counts pooling, activation, concatenation and batch-normalization operations as separate layers, so totals vary widely: VGG16's stack contains 19 layers, ResNet50 175, and DenseNet121 427. Unfreezing the last 4, 20 and 40 layers respectively therefore exposes a comparable upper portion of each model rather than an equal nominal number of layers. Earlier layers encode generic edges and textures that transfer well and were left frozen.

Results and discussion





First table summarizes the performance of the three architectures on the test set of 300 images. VGG16 performed best (accuracy 0.9167, F1 0.9186), followed by ResNet50 (0.9033, 0.9030) and DenseNet121 (0.8833, 0.8829). The differences are small in absolute terms: the models classified 275, 271 and 265 images correctly. With a test set of this size the 95% confidence interval on each figure spans roughly ± 3 percentage points, so the ranking should be regarded as indicative rather than conclusive. F1-score and accuracy are nearly identical for all three models, as expected on a balanced test set.

Performance was not related to model size. ResNet50 is the largest network at 24.1 million parameters yet ranked second, while DenseNet121 came within 3.3 percentage points of the best model using 7.3 million — under a third as many. This suggests the task is not limited by model capacity, and that DenseNet121 offers the best performance-to-size ratio where inference cost matters.

Conclusion and future work

All three models exceeded 88% test accuracy, with VGG16 performing best at 91.67%. The margins between models were small relative to the test set size, so the ranking is indicative rather than definitive. Accuracy was also largely independent of model size: DenseNet121 came within 3.3 percentage points of the best result using under a third of ResNet50's parameters.

Future work should train on the full corpus of 51,488 images, over more epochs and across several random seeds, to establish whether the observed differences are genuine. A larger test set or k-fold cross-validation would narrow the confidence intervals enough to compare models statistically.